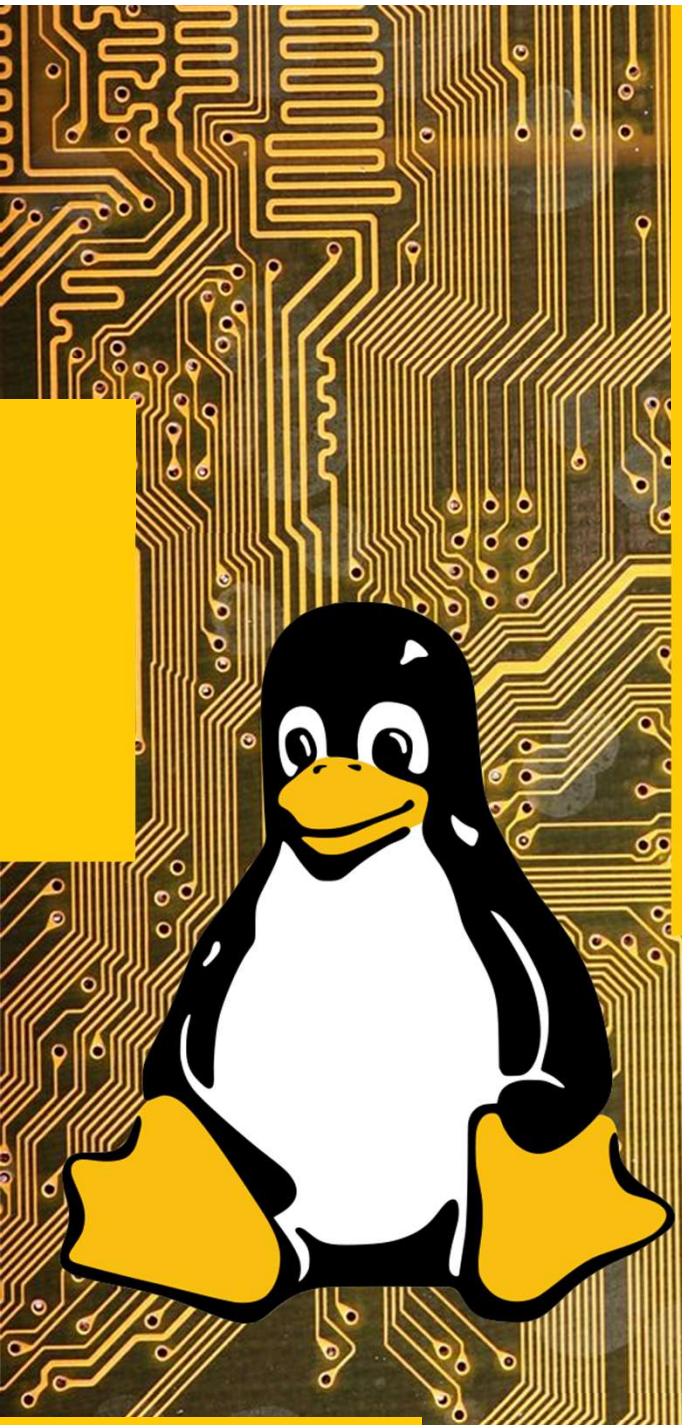


INHA UNIVERSITY IN TASHKENT

# SOC 3010 OPERATING SYSTEMS

## IPC III

### Concurrent Servers

**INSTRUCTOR:** Jalolliddin Yusupov

# Many Clients at Once

## TODAY: A CALL CENTRE

| | |
|---|---|
| Design 1 | **one operator** — one client at a time |
| Design 2 | **a new operator for every call** — a process per client |
| Design 3 | **a fixed team** — a pool of processes |
| Design 4 | **one operator, many phones** — select / poll (the idea only) |

### Last lecture's question

Client A connects and says nothing. Client B is put **on hold by the OS** — for as long as A's lunch lasts. Today we fix that.

**A server is judged by what happens to the second client.**

## Design 1: One Operator

Exercise

```
Server:
  L = listen(5000)
  loop forever:
      C = accept(L)
      serve(C)          /* takes 2 seconds */
      close(C)

serve(C):
  name = read(C)
  work for 2 seconds
  write(C, "done " + name)
```

Three clients, **A**, **B** and **C**, connect at the same moment (A a little first).

**(a)** After how many seconds does each one get its reply?

**(b)** While A is being served, where exactly are B and C?

**(c)** How busy is the CPU during these seconds?

# Exercise: Answer — 2, 4, 6

```
$ ./server_one &
$ ./timed_client localhost A &
$ ./timed_client localhost B &
$ ./timed_client localhost C &
client A: got "done A" after 2 s
client B: got "done B" after 4 s
client C: got "done C" after 6 s
```

**One operator: every client pays for all the clients in front of it.**

### (a) 2, 4 and 6 seconds

Each client waits for everybody in front of it. With 100 clients the last one waits 200 seconds.

### (b) In the OS's listen queue

B and C **are connected** — their `connect` returned at once, and they even sent their names. The OS is holding their calls; the server has simply not said `accept` yet.

### (c) The CPU is idle

The server is **asleep** almost the whole time (our "work" is a `sleep`; a real server would be waiting for a disk or a database). The clients wait for **nothing**.

```
Server:
  L = listen(5000)
  loop forever:
      C = accept(L)
      serve(C)
      close(C)

serve(C):
  name = read(C)
  work for 2 seconds
  write(C, "done " + name)
```

# Design 2: A New Process for Every Client

```
Server:
  L = listen(5000)
  loop forever:
      C = accept(L)
      if fork() == 0:  /* child  */
          close(L)
          serve(C)
          close(C)
          exit
      close(C)         /* parent */
```

### The call centre

The receptionist (the **parent**) picks up, **hires a new operator** for this caller ( `fork` ), hands over the call, and is back at the reception phone immediately. The operator (the **child**) serves one client and goes home ( `exit` ).

### Why it works: Week 1

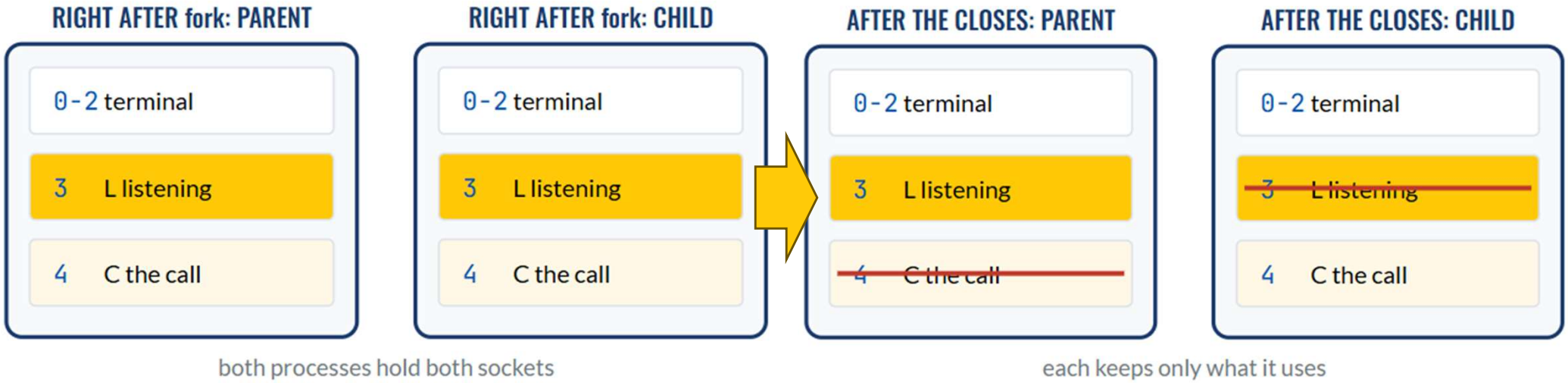
`fork` copies the descriptor table, so the child **inherits the open call** C. Parent and child both hold it — for a moment.

### Who makes them run at the same time?

The **OS scheduler** (Week 2). Three children = three processes; on a 4-core laptop they really run in parallel. You wrote no scheduling code at all.

**The parent only accepts; every client gets its own child process.**

# Draw It: After fork, Who Holds What?




**After fork: the parent closes the call, the child closes the listening socket.**

## The Sacred Rule of Closing, again

Week 3, pipes: *every process closes every end it will not use, immediately after fork.* Here: the **parent closes C** (the child is talking to that client), the **child closes L** (it will never accept).

## Why it is not just tidiness

A call ends only when **everyone** holding it has closed it. If the parent keeps C, the client never sees the end of the call — and the parent's table gains one descriptor per client until the OS refuses to give more.

```
Server:
  L = listen(5000)
  loop forever:
      C = accept(L)
      if fork() == 0:  /* child  */
          close(L)
          serve(C)
          close(C)
          exit
      close(C)         /* parent */
```

# Exercise: Three Broken Servers

For each one: name the bug, say **what you would see** (in the clients, in `ps`), and write the one-line fix.

SERVER X

```
loop forever:
  C = accept(L)
  if fork() == 0:
      close(L)
      serve(C)
      close(C)
      exit
  /* nothing here */
```

SERVER Y

```
loop forever:
  C = accept(L)
  if fork() == 0:
      serve(C)
      close(C)
      /* no exit */
  close(C)
```

SERVER Z

```
loop forever:
  C = accept(L)
  if fork() == 0:
      close(L)
      serve(C)
      close(C)
      exit
  close(C)
/* parent never
   waits for children */
```

**Each server works for the first client. What goes wrong after 1000 clients?**

| bug | what you see | fix |
|---|---|---|
| **X** — the P. never closes C | **Leaked descriptors.** Each client costs the parent one slot: 4, 5, 6 … Around the 1020th client the OS says Too many open files and accept fails. Also: clients never see the end of the call. | close(C) in the parent |
| **Y** — the Ch. never exits | After serving, the child **falls back into the loop** and calls accept itself. Now the children are receptionists too, and each of them forks again. ps shows the number of servers growing with every client. | exit at the end of the child |

# Exercise: Three Broken Servers

For each one: name the bug, say **what you would see** (in the clients, in `ps`), and write the one-line fix.

SERVER X

```
loop forever:
  C = accept(L)
  if fork() == 0:
      close(L)
      serve(C)
      close(C)
      exit
  /* nothing here */
```

SERVER Y

```
loop forever:
  C = accept(L)
  if fork() == 0:
      serve(C)
      close(C)
      /* no exit */
  close(C)
```

SERVER Z

```
loop forever:
  C = accept(L)
  if fork() == 0:
      close(L)
      serve(C)
      close(C)
      exit
  close(C)
/* parent never
   waits for children */
```

**Each server works for the first client. What goes wrong after 1000 clients?**

| bug | what you see | fix |
|---|---|---|
| **Z** — nobody waits for the children | **1000 zombies** (Week 1): finished children that nobody collected. ps shows <defunct>, state Z. | the SIGCHLD handler with the waitpid loop from Homework 1 |

### Three weeks of this course in one loop

X is Week 3 (close what you do not use). Y is Week 1 (what `fork` returns, and to whom). Z is Week 1 again (reap your children).

### Why servers make it worse

A normal program with these bugs ends, and the OS cleans up. A server runs for **months**: every small leak is multiplied by every client.

**In a server, every forgotten close, exit or wait is multiplied by the number of clients.**

# In C: The Process-per-Client Server

```c
int main(void) {
    struct sigaction sa;
    sa.sa_handler = reap;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGCHLD, &sa, NULL);

    int L = server_listen(5000);
    while (1) {
        int c = server_accept(L);
        if (fork() == 0) {              /* child: this client's own operator */
            close(L);                   /* the child never accepts           */
            serve(c);
            close(c);
            exit(0);
        }
        close(c);                       /* parent: the child has this call  */
    }
}
```

```
$ ./server_fork &
$ ./timed_client localhost A &
$ ./timed_client localhost B &
$ ./timed_client localhost C &
client A: got "done A" after 2 s
client B: got "done B" after 2 s
client C: got "done C" after 2 s
```

## Line by line = the pseudo-code

The first five lines install the zombie collector. The loop is exactly as before.

**`SA_RESTART` matters here**

When a child dies, the signal interrupts the parent's sleeping `accept`. `SA_RESTART` makes the OS start it again.

# Exercise: The Visitor Counter

```
visitors = 0                /* a global variable */

Server:
  L = listen(5000)
  loop forever:
      C = accept(L)
      if fork() == 0:
          close(L)
          visitors = visitors + 1
          name = read(C)
          write(C, "hello " + name +
                   ", you are visitor " + visitors)
          close(C)
          exit
      close(C)
```

Three clients connect **one after another**: Ali, then Vali, then Sami.

**What does each client see?**

(1) visitor 1, 2, 3

(2) visitor 1, 1, 1

(3) visitor 0, 0, 0

(4) different on every run

```
$ ./client localhost Ali
client: the server said "hello Ali, you are visitor 1"
$ ./client localhost Vali
client: the server said "hello Vali, you are visitor 1"
$ ./client localhost Sami
client: the server said "hello Sami, you are visitor 1"
```

## (2): everybody is visitor 1

fork gives the child a **copy** of all memory (Week 1). The child adds 1 to **its own copy** and exits; the parent's visitors is still 0 when the next child is born.

# Exercise: The Visitor Counter

```
visitors = 0              /* a global variable */

Server:
  L = listen(5000)
  loop forever:
      C = accept(L)
      if fork() == 0:
          close(L)
          visitors = visitors + 1
          name = read(C)
          write(C, "hello " + name +
                   ", you are visitor " + visitors)
          close(C)
          exit
      close(C)
```

### Fix A: one line

Count in the **parent**, before `fork` : `visitors = visitors + 1` right after `accept` . Each child is born with the correct number.

## The price of Design 2

A process per client means **private memory per client**. Safe — one client cannot damage another — but anything shared must be asked for.

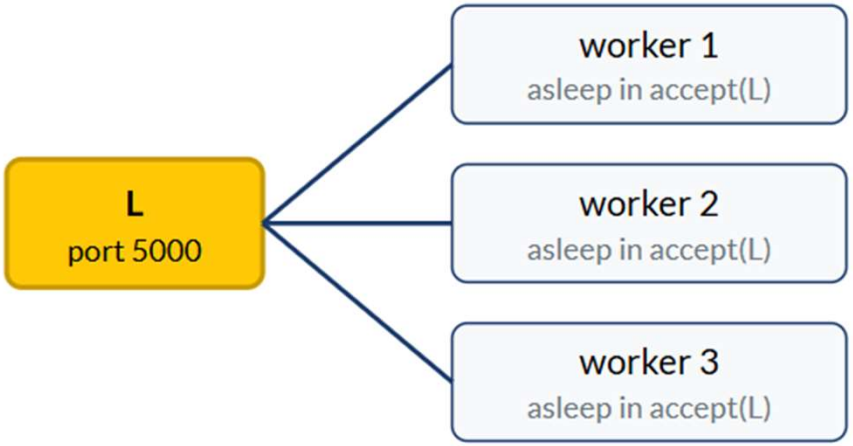
### Fix B: Week 4

If the **children** must change shared data (a chat room, a score table): put it in **shared memory** ( `shared_alloc` , before `fork` ) — and protect it with a **semaphore**, because now two children really can do `counter++` at the same time.

**Children do not share variables: fork copies, it does not connect.**

# Design 3: A Fixed Team (a Pool of Processes)

```
Server:
  L = listen(5000)
  repeat N times:              /* hire the team ONCE */
      if fork() == 0:
          loop forever:
              C = accept(L)
              serve(C)
              close(C)
  wait for the children        /* parent does nothing else */
```




### How it works

All N workers sleep in `accept` on the **same** listening socket (they inherited it). For each caller **the OS wakes exactly one** of them. No receptionist is needed.

### Sounds familiar?

"At most N inside, the others wait" is last week's **counting semaphore** that starts at N.

| | |
|---|---|
| Design 2 | a `fork` for every client; 10,000 clients = 10,000 processes: memory and the scheduler suffer |
| Design 3 | no `fork` per client; never more than N processes; but client N+1 **waits** |

**A pool trades a little waiting for a fixed, predictable cost.**

# Exercise: Five Clients, Two Seconds Each

Clients **A, B, C, D, E** connect at the same moment, in that order.
Every request needs **2 seconds** of work.
Fill in when each client gets its reply.

| server | A | B | C | D | E | done | processes |
|---|---|---|---|---|---|---|---|
| Design 1: one at a time | | | | | | | |
| Design 2: a process per client | | | | | | | |
| Design 3: a pool of 2 | | | | | | | |
| Design 3: a pool of 5 | | | | | | | |

**(a)** Fill in the table (seconds).

**(b)** How many server processes exist at the busiest moment, in each row?

**(c)** Which design would you choose for 5 clients a day? For 50,000 at once?

```
$ ./server_pool &        # POOL = 2, then five clients
client A: got "done A" after 2 s
client B: got "done B" after 2 s
client C: got "done C" after 4 s
client D: got "done D" after 4 s
client E: got "done E" after 6 s
```

**More processes, less waiting — until the processes themselves become the problem.**

# Design 4: One Operator, Many Phones (select / poll)

```
Server:
  L = listen(5000)
  calls = [ L ]                  /* every phone on my desk */
  loop forever:
      ready = poll(calls)        /* sleep until at least
                                    ONE phone rings      */
      for each phone in ready:
          if phone == L:
              add accept(L) to calls
          else:
              answer that client a little
```

**One process can serve many clients if the OS tells it which one is ready.**

## The new idea

Instead of sleeping on **one** descriptor, the process asks the OS: **"wake me when any of these has something for me."** `select` and `poll` are the two classic calls that do it.

## Who works like this

**nginx**, **Redis**, **Node.js** — with a faster relative of poll called epoll. We come back to it in Week 11, once we have threads to compare it with.

| | |
|---|---|
| no `fork` at all | one process, thousands of clients |
| memory is shared for free | the visitor counter simply works — and no semaphore is needed, because only one thing runs at a time |
| the danger | one slow client job (our 2 seconds of work) blocks **everybody**: the operator must only do short things |

# Four Designs, Side by Side

| design | processes | who waits | shared data? | cost | used by |
|---|---|---|---|---|---|
| **1 · one at a time** | 1 | each waits for all before it | trivially (one process) | simplest code | small tools, tests |
| **2 · process per client** | 1 + one per client | nobody waits | no — needs shared memory + semaphore | a `fork` per client; unlimited processes | PostgreSQL, sshd |
| **3 · pool of N** | 1 + N | client N+1 waits | no — same as 2 | fixed and predictable | Apache (prefork), PHP-FPM |
| **4 · select / poll** | 1 | nobody waits, if every step is short | yes, for free | harder to write; one slow step blocks all | nginx, Redis, Node.js |

**Choose by asking: how many clients, how long is each job, what must they share?**

### There is no best design

There is a **trade-off**: simplicity, waiting time, memory, safety between clients. An engineer chooses; an exam asks you to **justify** the choice.

### After the midterm

Week 9 adds a fifth design: **a thread per client** — cheaper than a process, sharing memory by default, and therefore needing every lock you learned in Week 4.

# What the OS Did for You This Week

| the OS … | so that … |
|---|---|
| gave you a **socket as a file descriptor** | so `read`, `write`, `close` work on a network exactly as on a file or a pipe (Week 3) |
| kept the **port table** | delivered every arriving byte to the one process that owns the port |
| kept the **listen queue** | answered the phone while your server was busy, and held the callers |
| **put processes to sleep and woke them** | in `accept` and `read`: no CPU wasted while waiting (Week 2: states) |
| **buffered** the bytes in both directions | a fast writer is made to wait for a slow reader, as with a full pipe (Week 3) |
| **copied the descriptor table** in `fork` | so a child could inherit a live phone call (Week 1) |
| **scheduled** one process per client | ran them at the same time on your CPU cores (Week 2) |
| woke **exactly one** pool worker per caller | so two workers never serve the same client |
| cleaned up when a process died | closed its sockets, so the other side reads 0 instead of waiting forever |

**Your server is twenty lines because the operating system is twenty million.**



# Questions?